

Activity 4: Application Architecture Design Using the C4 Model

Sample Solution for PUP IT Asset Management Transformation

Introduction

This sample solution demonstrates how to complete Activity 4 deliverables using the PUP IT Asset Management transformation as the context. The solution builds upon the TO-BE business architecture (Activity 2), the data architecture (Activity 3), and the technology architecture planning (Activity 5 preview). The C4 model is applied to document the Unified Integration & Intelligence Layer, the Polyglot Persistence Layer, and the AI Analytics & Decision Support components shown in the TO-BE diagram.

For this sample solution, we will assume a fictional company called **PUP IT Asset Management System (PUP-ITAMS)**, which manages IT assets across multiple campuses, serving faculty, staff, and students.

Deliverable 1: Level 1 - Context Diagram

System Definition

The central system being modeled is the **Unified Integration & Intelligence Layer** for PUP-ITAMS. This system encompasses:

- **IT Asset Management Core (Snipe-IT based)** : Tracks all IT assets including computers, peripherals, software licenses, and network equipment
- **Polyglot Persistence Layer**: Manages diverse data types using multiple database technologies

- **AI Analytics & Decision Support:** Provides predictive maintenance, demand forecasting, and anomaly detection

Actors (Users)

Actor	Description	Interactions
Faculty/Staff	University employees who request and use IT assets	Request equipment, view assigned assets, report issues, check asset status
Student	Students who may be issued loaner devices or lab equipment	Request loaner devices, view assigned equipment, report issues
Department Chair	Academic department heads who approve requests	Approve/reject asset requests, view department asset inventory
IT Help Desk	Tier 1 support staff	Check-in/out assets, update status, basic troubleshooting, assist users
IT Asset Manager	Specialized asset management team	Configure system, manage asset lifecycle, generate reports, oversee compliance
Property and Supply Office	University property custodians	Oversee asset accountability, approve disposals, maintain property records
Network Administrator	IT staff managing campus network infrastructure	View network relationships, configure auto-discovery, troubleshoot connectivity

Actor	Description	Interactions
Finance Staff	Budget and accounting personnel	View asset values, track depreciation, allocate costs to departments
Campus Administrator	University leadership	View executive dashboards, monitor KPIs, approve major procurements

External Systems

External System	Type	Data Exchange	Direction
PUP-HRIS	HR System	Employee data (status, department, location, position)	Inbound only
PUP-SIS (Student Information System)	Student System	Student enrollment status, course information	Inbound only
PUP Procurement System	ERP	Purchase orders, vendor data, costs, delivery dates	Inbound only
PUP Finance System (ERP)	Financial	Depreciation data, cost allocations, budget information	Outbound only
MDM (JAMF Pro for Mac, Intune for Windows)	Device Management	Telemetry data, compliance status, application inventory	Bidirectional

Advanced System Design and Implementation

Campus Network Scanner	Infrastructure	Asset-to-network mappings, switch ports, access points	Inbound only
Vendor Portals (Dell, HP, Microsoft)	External	Warranty claims, repair status, license management	Bidirectional
PUP Email / Microsoft Teams	Communication	Notifications, alerts, approval requests	Outbound only
Active Directory / Azure AD	Identity	Authentication, user groups, role assignments	Inbound only
CCTV / Security System	Security	Asset location tracking, theft alerts	Inbound only

Data Flows

Flow Name	Source	Destination	Content	Direction	Frequency
Employee Data	PUP-HRIS	System	Employee ID, Name, Department, Position, Status, Location	Inbound	Real-time (webhook)
Student Data	PUP-SIS	System	Student ID, Name, College, Year Level, Enrollment Status	Inbound	Daily batch

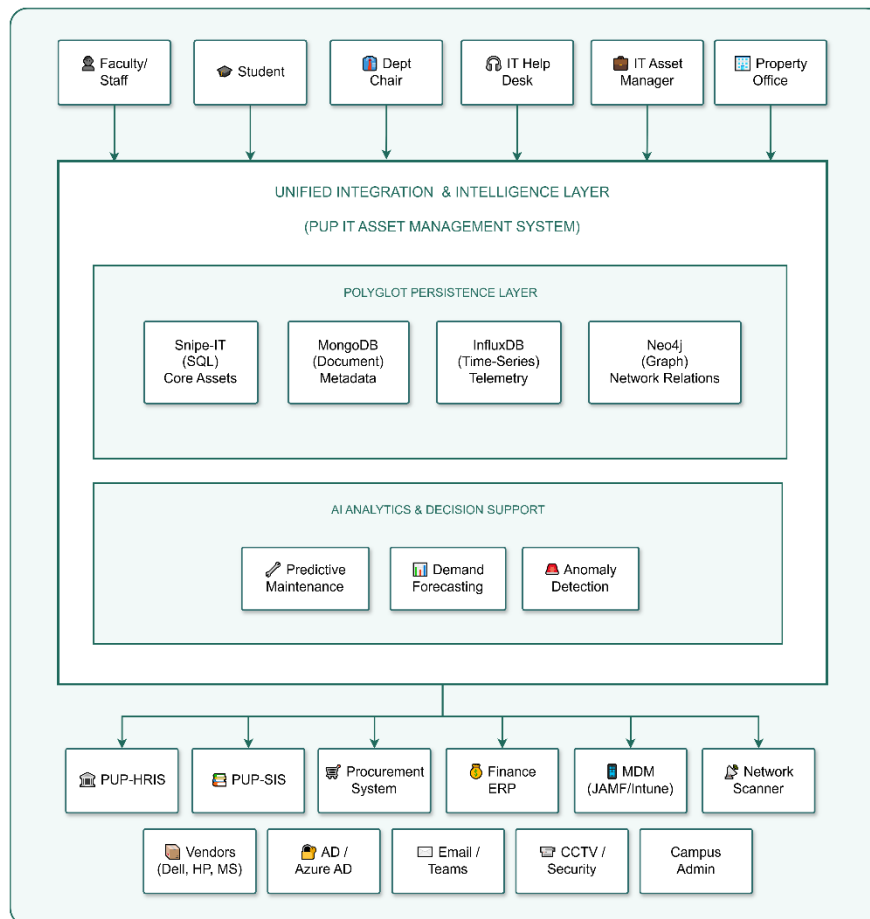
Advanced System Design and Implementation

Termination Event	PUP-HRIS	System	Employee ID, Termination Date, Last Working Day, Clearance Status	Inbound	Real-time
PO Data	Procurement System	System	PO Number, Vendor, Items, Costs, Expected Delivery Date	Inbound	Daily batch
Asset Depreciation	System	Finance ERP	Asset ID, Cost, Age, Current Value, Depreciation Schedule	Outbound	Monthly
Telemetry Data	MDM (JAMF/Intune)	System	CPU Temp, Battery Health, Disk Usage, Memory, Last Backup, Location	Inbound	Continuous (15-min intervals)
Health Alerts	System	IT Staff	Asset ID, Failure Probability (%), Recommendation, Urgency	Outbound	On detection
Network Map	Network Scanner	System	Asset ID, Switch Port, AP Name, VLAN, IP Address, Last Seen	Inbound	Daily
Warranty Claim	System	Vendor Portal	Asset ID, Serial Number, Issue	Outbound	As needed

Advanced System Design and Implementation

			Description, Diagnostic Report		
Claim Status	Vendor Portal	System	Claim ID, Status, Estimated Resolution, Shipping Tracking	Inbound	As needed
Notification	System	Email/Teams	User ID, Message Type, Action Required, Link	Outbound	Per event

Context Diagram



Deliverable 2: Level 2 - Container Diagram

Container Identification

Container	Type	Technology	Purpose
Web Application	SPA	React.js	Admin interface for IT staff, managers, property office, campus admin
Mobile Application	Mobile	React Native	Field operations for scanning, check-in/out, inventory audit
API Gateway	Service	Kong (self-managed on EKS)	Central entry point, authentication, rate limiting, request routing
API Application	Service	Python/Flask	Business logic, orchestration, REST/GraphQL endpoints
Asset Synchronization Service	Service	Python	CDC from Snipe-IT to other stores (Elasticsearch, Neo4j)
Telemetry Ingestion Service	Service	Python/Go	Real-time ingestion of health data from MDM (JAMF/Intune)
Predictive Analytics Service	Service	Python	AI/ML model serving for failure prediction and demand forecasting

Container	Type	Technology	Purpose
Workflow Orchestrator	Service	Temporal	Workflow management for offboarding, procurement, approvals
PostgreSQL Database	Database	PostgreSQL 14+	Core asset records (Snipe-IT), assignments, users
Document Store	Database	MongoDB	Unstructured metadata, warranty PDFs, configuration files, images
Time-Series Database	Database	InfluxDB	Telemetry data (CPU, battery, disk, memory, location history)
Graph Database	Database	Neo4j	Network relationships, department hierarchies, asset dependencies
Cache	Cache	Redis	Session storage, prediction caching, rate limiting, distributed locks
Message Broker	Queue	Kafka	Event distribution, CDC events, notifications
Search Engine	Search	Elasticsearch	Asset search, full-text indexing, log analytics
Batch Processor	Batch	Apache Airflow	Nightly reports, data exports, ETL jobs

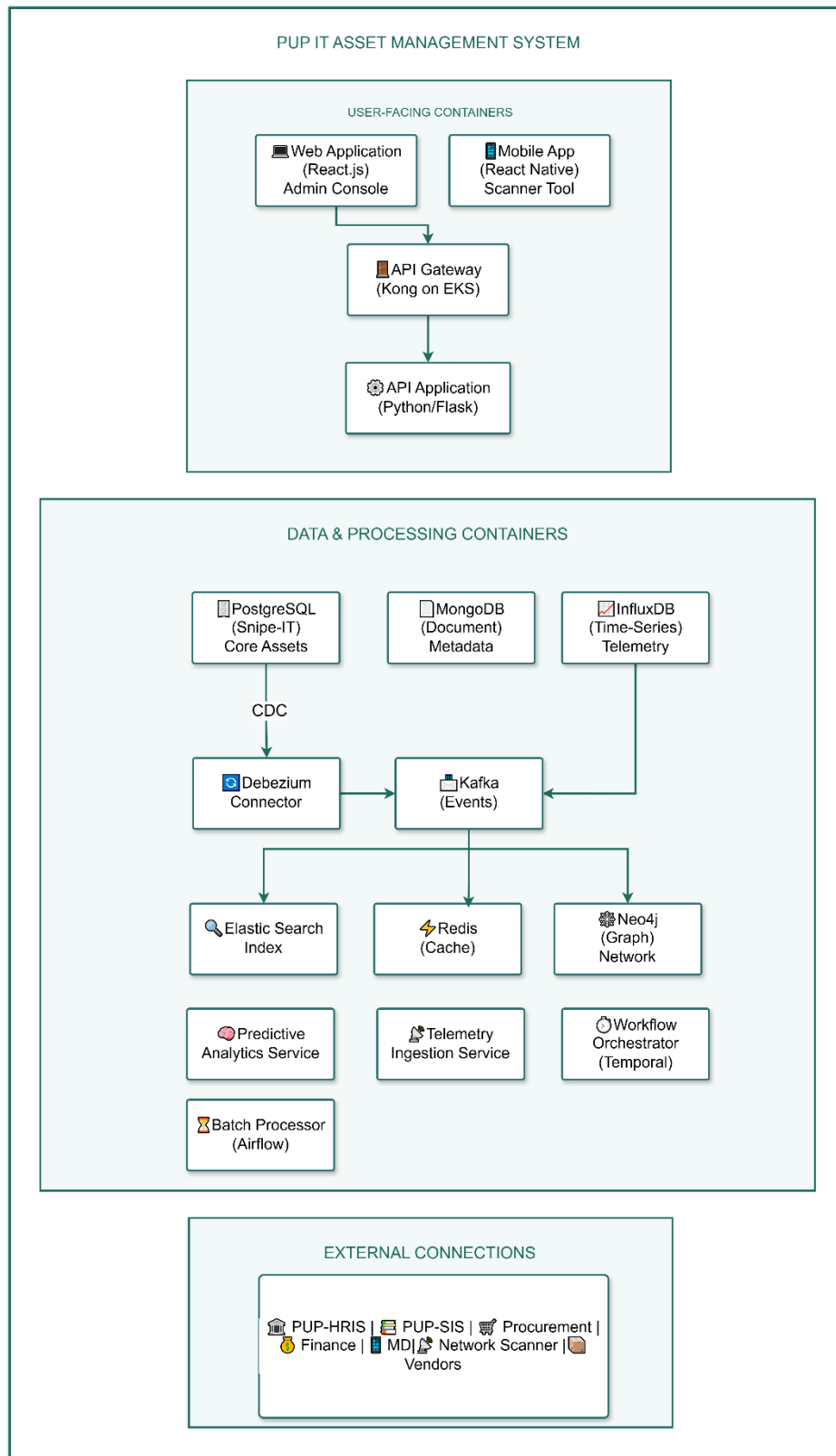
Container Communication

Source Container	Destination Container	Protocol	Purpose
Web App	API Gateway	HTTPS	All user interactions
Mobile App	API Gateway	HTTPS	Scanning, check-in/out
API Gateway	API Application	HTTPS	Authenticated requests
API Application	PostgreSQL	JDBC/ORM	CRUD operations for assets, assignments
API Application	MongoDB	Native Driver	Product catalog, metadata
API Application	Redis	Redis Protocol	Session management, caching
API Application	Kafka	Kafka Protocol	Event publishing
API Application	Elasticsearch	HTTP/REST	Search queries
API Application	Neo4j	Bolt Protocol	Network relationship queries
Asset Sync Service	PostgreSQL	CDC (Debezium)	Change capture
Asset Sync Service	Kafka	Kafka Protocol	Event publishing

Advanced System Design and Implementation

Source Container	Destination Container	Protocol	Purpose
Telemetry Service	MDM (JAMF/Intune)	HTTPS/REST	Pull telemetry data
Telemetry Service	InfluxDB	InfluxDB API	Store time-series data
Predictive Service	InfluxDB	InfluxDB API	Query historical telemetry
Predictive Service	Kafka	Kafka Protocol	Consume events, publish alerts
Predictive Service	PostgreSQL	JDBC	Update asset health scores
Workflow Service	PostgreSQL	JDBC	State persistence
Workflow Service	Kafka	Kafka Protocol	Publish workflow events
Batch Processor	PostgreSQL	JDBC	Query for exports
Batch Processor	S3	S3 API	Store exported files

Container Diagram



Container Descriptions Table

Container	Technology	Purpose	Key Features	Deployment
Web Application	React.js	Admin interface	Dashboard, asset search, reports, approval workflows, role-based views	S3 + CloudFront
Mobile App	React Native	Field operations	QR scanning, offline capability, location updates, barcode generation	App Store, Play Store
API Gateway	Kong	Entry point	Authentication (OAuth2/JWT), rate limiting, request routing, logging	EKS (managed)
API Application	Python/Flask	Business logic	REST/GraphQL endpoints, orchestration, validation, event publishing	EKS (scalable)
PostgreSQL	PostgreSQL 14+	Core asset data	ACID compliance, complex queries,	RDS Multi-AZ

Container	Technology	Purpose	Key Features	Deployment
			JSON support, full-text search	
MongoDB	MongoDB 6+	Unstructured metadata	Flexible schema for warranty docs, configs, images, audit logs	MongoDB Atlas
InfluxDB	InfluxDB 2.x	Telemetry storage	Time-series optimization, high write throughput, downsampling	InfluxDB Cloud
Neo4j	Neo4j 5.x	Network relationships	Graph traversal, relationship mapping, path finding	Neo4j Aura
Redis	Redis 7+	Caching	Session storage, prediction cache, rate limiting, distributed locks	ElastiCache
Kafka	Kafka 3.x	Event streaming	CDC events, notifications, audit trails, exactly-once semantics	MSK

Container	Technology	Purpose	Key Features	Deployment
Elasticsearch	Elasticsearch 8+	Search	Full-text search, asset lookup, log analytics, aggregations	Elastic Cloud
Debezium	Debezium 2.x	CDC	PostgreSQL change capture, schema evolution, event routing	Kafka Connect
Predictive Service	Python/Flask	AI inference	Model serving, predictions, A/B testing, model versioning	EKS
Telemetry Service	Python/Go	Data ingestion	MDM integration, telemetry processing, anomaly detection	EKS
Workflow Engine	Temporal	Orchestration	Order fulfillment, offboarding, approval workflows, compensation	Temporal Cloud

Container	Technology	Purpose	Key Features	Deployment
Batch Processor	Airflow	Scheduled jobs	Reports, exports, syncs, data quality checks	MWAA

Deliverable 3: Level 3 - Component Diagram (API Application)

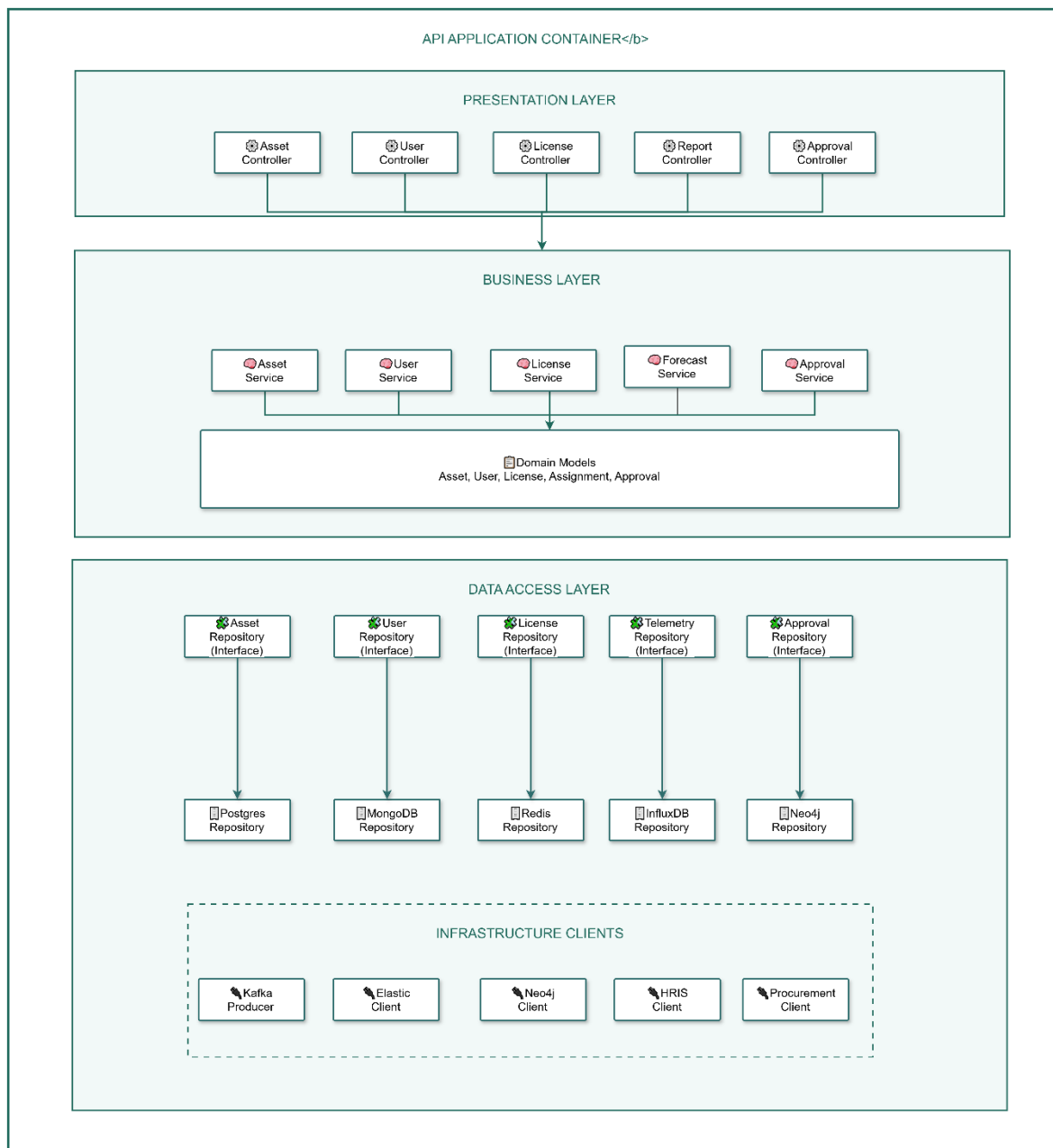
Component Identification

Component	Type	Responsibility
Asset Controller	HTTP Layer	Handle asset-related requests, validation, response formatting
User Controller	HTTP Layer	Handle user-related requests, profile management
License Controller	HTTP Layer	Handle license-related requests, compliance checking
Report Controller	HTTP Layer	Handle report generation requests
Approval Controller	HTTP Layer	Handle approval workflows
Asset Service	Business Logic	Asset lifecycle management, business rules, coordination
User Service	Business Logic	User management, role resolution, department mapping

Component	Type	Responsibility
License Service	Business Logic	License allocation, compliance monitoring, renewal tracking
Forecasting Service	Business Logic	Demand prediction, failure forecasting
Approval Service	Business Logic	Approval routing, notification, deadline management
Asset Entity	Domain	Asset domain model with business behavior
User Entity	Domain	User domain model
Assignment Entity	Domain	Assignment domain model
Asset Repository (Interface)	Data Access	Abstraction for asset data access
User Repository (Interface)	Data Access	Abstraction for user data access
Postgres Asset Repository	Data Access	PostgreSQL CRUD operations
MongoDB Asset Repository	Data Access	Metadata operations
Redis Cache Repository	Data Access	Cache operations
Elasticsearch Client	Infrastructure	Search operations
Neo4j Client	Infrastructure	Graph operations
Kafka Event Publisher	Integration	Publish events to Kafka

Component	Type	Responsibility
HRIS Client	Integration	Call HRIS external API
Procurement Client	Integration	Call Procurement external API

Layered Architecture Diagram



Component Descriptions

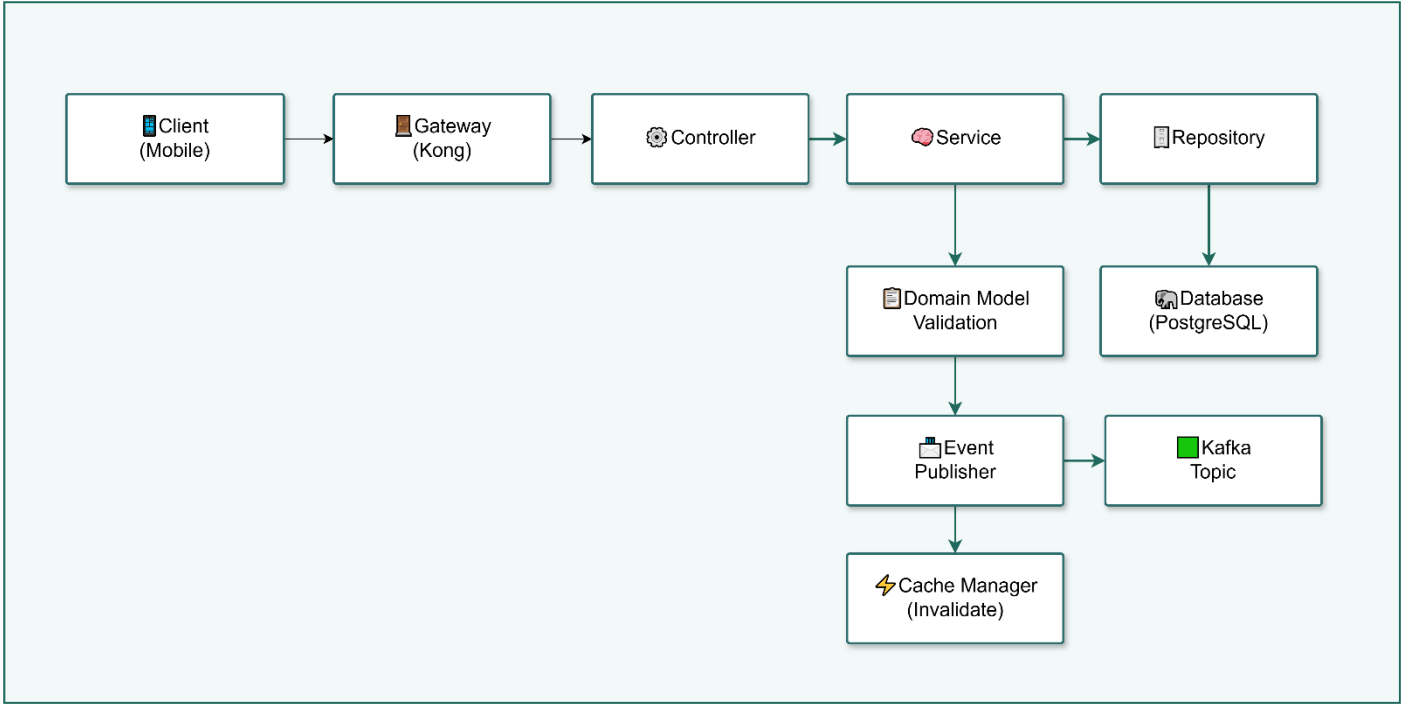
Component	Inputs	Outputs	Key Responsibilities	Dependencies
Asset Controller	HTTP Request (JSON)	HTTP Response	Validate input, authenticate user, route to service, format response	Asset Service
Asset Service	Domain objects (Asset, User ID)	Domain objects (Asset, Assignment)	Business logic, rule enforcement (no duplicate assignment), coordination, transaction management	Asset Repo, User Repo, Assignment Repo, Event Publisher, Cache Manager
Asset Repository (Interface)	Query parameters, Asset object	Asset object, List<Asset>	Abstraction for data access - defines contract	None (interface)
Postgres Asset Repository	Asset object, Query params	Asset object, List<Asset>	PostgreSQL CRUD operations, complex queries, transaction support	SQLAlchemy ORM, PostgreSQL connection pool

Component	Inputs	Outputs	Key Responsibilities	Dependencies
MongoDB Asset Repository	Asset ID, Metadata	Asset metadata, Document	Unstructured metadata storage, warranty documents, configuration files	MongoDB driver
Redis Cache Repository	Cache key, Value	Cached value, Boolean	Cache read/write, TTL management, invalidation	Redis client
Elasticsearch Client	Search query string	Search results list	Full-text search across asset names, descriptions, tags	Elasticsearch client
Kafka Event Publisher	Domain event object	Delivery acknowledgment	Publish events (AssetCreated, AssetAssigned, AssetRetired) to Kafka topics	Kafka producer
Domain Models (Asset, Assignment)	Service layer calls	Domain objects, Validation results	Encapsulate business behavior, enforce invariants,	None

Component	Inputs	Outputs	Key Responsibilities	Dependencies
			calculate derived fields	

Component Interaction Flows

Flow 1: Asset Assignment

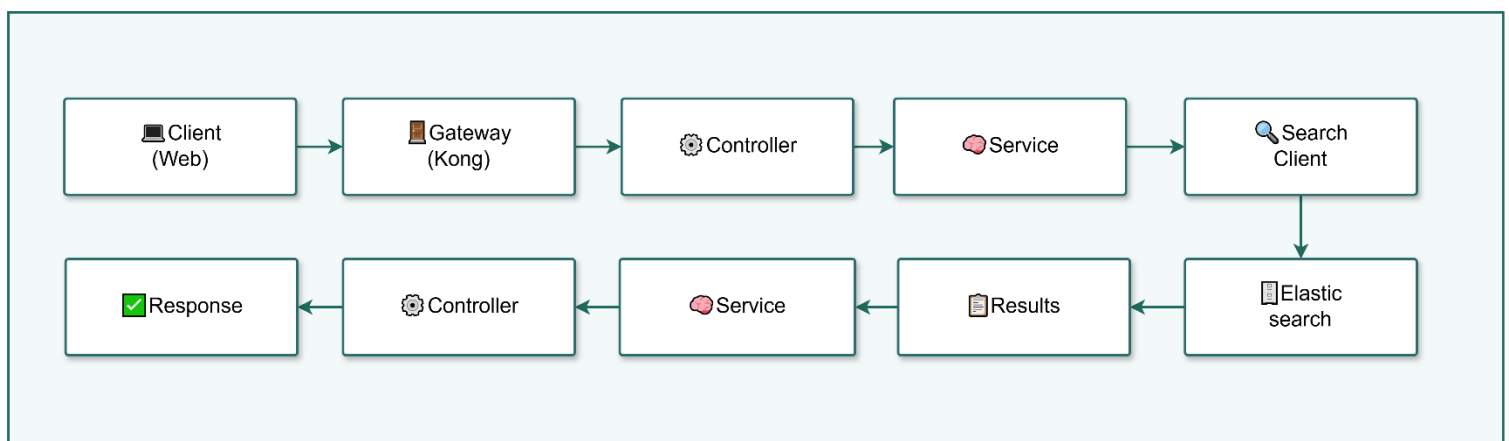


Step-by-step description:

1. Mobile app sends POST request to `/api/assets/{id}/assign` with user ID in body
2. API Gateway authenticates JWT token, applies rate limiting, routes to Asset Controller
3. Asset Controller validates request body (user ID present, asset ID valid format)
4. Asset Controller calls Asset Service's `assign_asset(asset_id, user_id)` method
5. Asset Service calls User Repository to verify user exists and is active
6. Asset Service calls Asset Repository to fetch current asset

7. Asset Service calls Domain Model's `can_assign_to(user)` to validate business rules
8. Asset Service updates asset status, calls Asset Repository to save changes
9. Asset Service calls Assignment Repository to create assignment history record
10. Asset Service calls Event Publisher to emit `AssetAssigned` event to Kafka
11. Asset Service calls Cache Manager to invalidate asset cache keys
12. Asset Service returns updated Asset and Assignment to Controller
13. Controller formats JSON response and returns HTTP 200

Flow 2: Asset Search

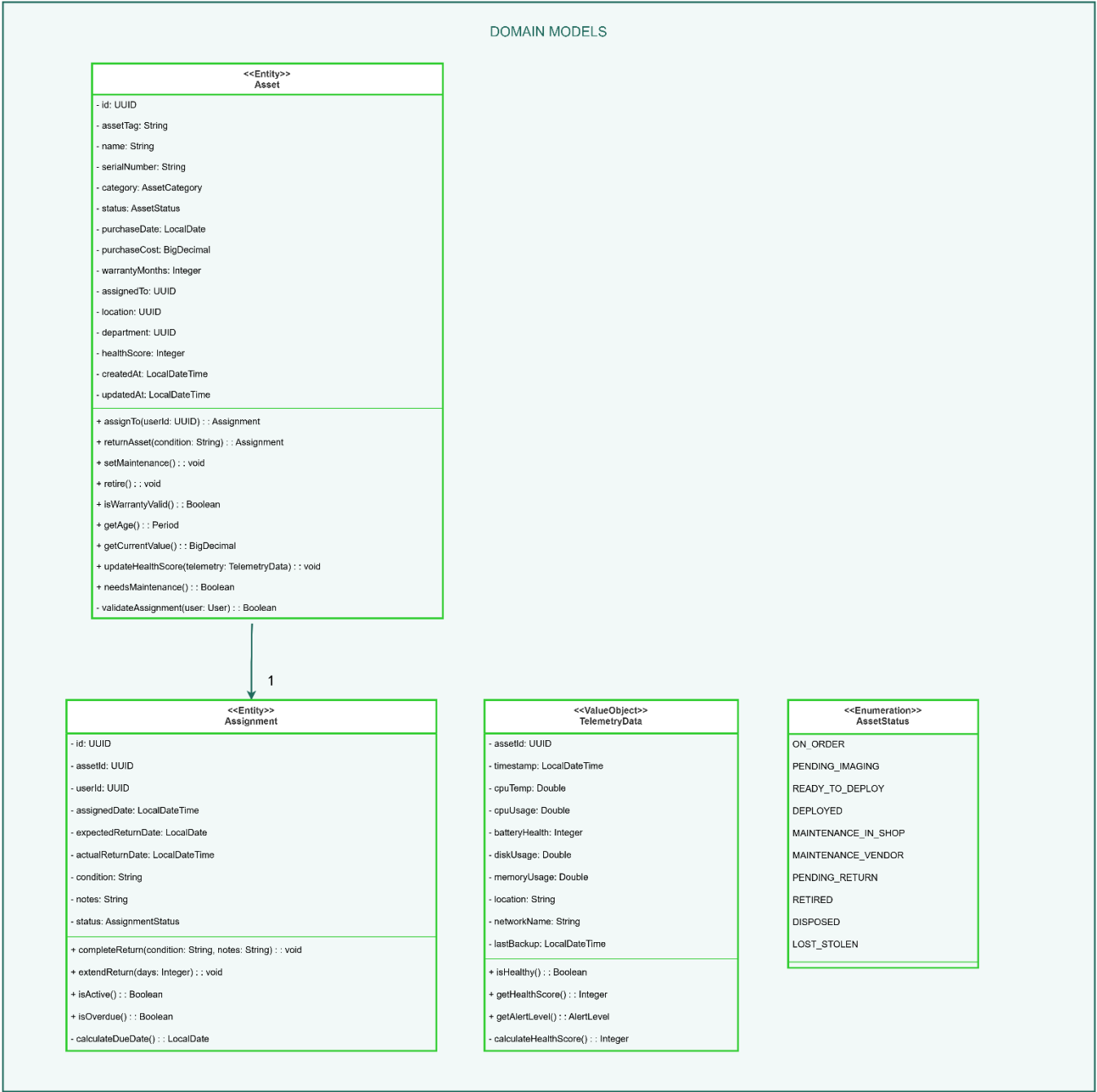


Step-by-step description:

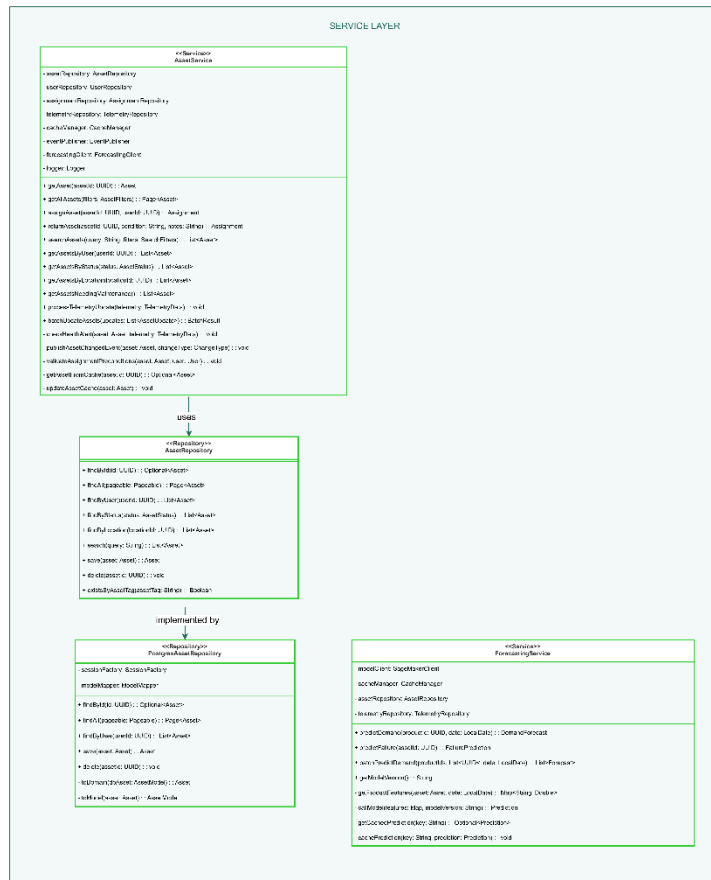
1. Web app sends GET request to `/api/assets/search?q=laptop`
2. API Gateway authenticates, routes to Asset Controller
3. Asset Controller validates query parameter (non-empty, max length 100)
4. Asset Controller calls Asset Service's `search_assets(query)` method
5. Asset Service calls Elasticsearch Client with search query
6. Elasticsearch Client returns matching asset IDs with relevance scores
7. Asset Service optionally fetches full asset details from PostgreSQL (if needed)
8. Asset Service returns list of matching assets to Controller
9. Controller formats JSON response with search results and metadata

Deliverable 4: Level 4 - Code (UML Class Diagrams)

Asset Entity Class Diagram



Asset Service Class Diagram



Deliverable 5: Architecture Decision Records (ADRs)

ADR-001: Polyglot Persistence Strategy

Field	Value
Title	Use Polyglot Persistence with Multiple Database Technologies
Status	Accepted
Context	The PUP IT Asset Management System must handle diverse data types: core asset records (relational with ACID requirements), unstructured metadata (warranty PDFs, configuration files, images), real-time telemetry (time-series data from thousands of devices), and network relationships (graph data for asset-to-network mappings). A single database cannot optimally serve all these use cases.
Decision	Adopt polyglot persistence using: PostgreSQL (Snipe-IT) for core asset records with ACID compliance, MongoDB for unstructured metadata and flexible schemas, InfluxDB for time-series telemetry data, Neo4j for graph-based network relationships and department hierarchies. Use Debezium for CDC synchronization between databases.
Alternatives Considered	1. Single relational database (PostgreSQL only) - rejected because time-series performance would be poor and flexible schema for metadata would require complex EAV patterns. 2. Single document database (MongoDB only) - rejected because ACID transactions across multiple documents are weak and complex reporting queries would be difficult. 3. Single time-series database only - rejected because relational integrity would be lost.

Field	Value
Consequences	Positive: Optimal performance for each use case; flexible schemas for evolving metadata; high write throughput for telemetry; efficient graph traversal for network queries. Negative: Increased operational complexity (multiple database types to manage); eventual consistency challenges across databases; need for CDC infrastructure (Debezium + Kafka); additional expertise required for each database type.
Related Decisions	ADR-002 (Event-Driven Synchronization), ADR-003 (CDC Strategy)
Date	2025-03-15

ADR-002: Event-Driven AI Integration Pattern

Field	Value
Title	Use Event-Driven Pattern for Predictive Maintenance and Analytics
Status	Accepted
Context	Predictive maintenance requires analyzing telemetry data from over 5,000 university IT assets (faculty/staff computers, lab machines, network equipment). Real-time analysis for every telemetry point is not required; predictions can be generated asynchronously. The system must handle bursts of telemetry data without overwhelming downstream services.
Decision	Implement event-driven AI using Kafka as the central message bus. Telemetry data from MDM (JAMF/Intune) is published to Kafka topics. The Predictive Analytics Service consumes these events asynchronously, runs ML models (SageMaker endpoints), and publishes alerts back to

Field	Value
	Kafka for downstream processing (notifications, ticket creation, asset updates).
Alternatives Considered	1. Request-reply pattern (synchronous API calls from telemetry service to AI) - rejected: would require synchronous calls for every telemetry update (5,000 assets × 96 readings/day = 480,000 synchronous calls), creating high latency and load. 2. Batch processing only (nightly batch predictions) - rejected: would miss real-time anomaly detection for critical failures (e.g., sudden battery failure, overheating). 3. Serverless functions (Lambda triggered by SQS) - rejected: cold starts would impact processing latency for burst traffic.
Consequences	Positive: Decoupled architecture - telemetry ingestion independent of AI processing; resilient - if AI service fails, events persist in Kafka for replay; scalable - consumer group can scale horizontally with load. Negative: Eventual consistency - alerts may be delayed by seconds to minutes; additional infrastructure complexity (Kafka topics, consumer groups, offset management); debugging distributed flows more challenging.
Related Decisions	ADR-001 (Polyglot Persistence), ADR-004 (Telemetry Ingestion Strategy)
Date	2025-03-16

ADR-003: API Gateway Selection

Field	Value
Title	Use Kong as API Gateway Instead of AWS API Gateway

Field	Value
Status	Accepted
Context	The system needs an API gateway to handle authentication (OAuth2/JWT), rate limiting, request routing, logging, and potentially plugin-based transformations. The system is deployed on AWS EKS. Options include AWS-native API Gateway and open-source Kong (self-managed or on EKS).
Decision	Use Kong API Gateway deployed on the same EKS cluster as the API application. Kong will handle JWT validation, rate limiting per user/endpoint, request/response logging, and route traffic to the API application service.
Alternatives Considered	1. AWS API Gateway - rejected due to vendor lock-in; limited plugin ecosystem; higher cost at scale; less flexible for custom authentication flows. 2. NGINX with Lua scripts - rejected because custom scripting would be needed for JWT validation and rate limiting, increasing maintenance burden. 3. Traefik - rejected due to smaller plugin ecosystem and community support.
Consequences	Positive: No vendor lock-in; extensive plugin ecosystem (OAuth2, rate limiting, transformations); runs on same Kubernetes infrastructure as API; open source with enterprise options. Negative: Self-managed on EKS (operational overhead for updates, scaling, monitoring); requires additional configuration for high availability.
Related Decisions	ADR-005 (Container Orchestration with Kubernetes)
Date	2025-03-17

ADR-004: Telemetry Ingestion Architecture

Field	Value
Title	Use Pull-Based Telemetry Ingestion from MDM Solutions
Status	Accepted
Context	The system must ingest telemetry data (CPU temperature, battery health, disk usage, location) from MDM solutions: JAMF Pro for Mac devices and Microsoft Intune for Windows devices. Both MDM solutions provide REST APIs for querying device inventory and status. The system has ~3,000 Mac devices and ~2,000 Windows devices.
Decision	Implement a pull-based ingestion service that periodically queries MDM APIs at configurable intervals (15 minutes for standard telemetry, 5 minutes for high-value assets). The service stores raw telemetry in InfluxDB and publishes health alerts to Kafka when thresholds are exceeded. For location tracking, the service uses the MDM's reported location data (based on Wi-Fi triangulation).
Alternatives Considered	1. Push-based (webhooks from MDM) - rejected because MDM webhook support is inconsistent across platforms (JAMF supports limited webhooks, Intune requires additional configuration). 2. Agent-based (custom agent on each device) - rejected because deploying and maintaining agents on 5,000 devices across campus is operationally complex and would require additional security approvals. 3. Third-party telemetry platform - rejected due to cost and data residency concerns.
Consequences	Positive: Consistent pull mechanism across both MDM platforms; configurable intervals to balance freshness vs. API rate limits; no client-side agent deployment required; works within existing MDM infrastructure. Negative: Pull-based means telemetry is not real-time

Field	Value
	(up to 15-minute delay); increased API call volume (~480,000 calls per day); need to manage API rate limits and pagination.
Related Decisions	ADR-001 (Polyglot Persistence - InfluxDB), ADR-002 (Event-Driven AI)
Date	2025-03-18

ADR-005: Container Orchestration with Kubernetes

Field	Value
Title	Use AWS EKS (Elastic Kubernetes Service) for Container Orchestration
Status	Accepted
Context	The system consists of multiple microservices (API Application, Predictive Service, Telemetry Service, Sync Service) that need to be deployed, scaled, and managed. The team needs portability, auto-scaling, service discovery, and rolling updates. The infrastructure is primarily on AWS.
Decision	Deploy all stateless services on AWS EKS (Elastic Kubernetes Service). Use managed node groups with auto-scaling based on CPU/memory metrics. Use Horizontal Pod Autoscaler (HPA) to scale individual services based on load. Use AWS Load Balancer Controller for ingress.
Alternatives Considered	1. AWS ECS (Elastic Container Service) - rejected because ECS is AWS-specific, limiting portability; less ecosystem support for custom operators and CRDs; fewer community resources. 2. Self-managed Kubernetes on EC2 - rejected due to operational overhead of managing control plane (etcd, API server, scheduler). 3. Serverless

Field	Value
	(Lambda) - rejected because services are long-running and have steady-state traffic; Lambda would be more expensive at scale and has execution time limits.
Consequences	Positive: Portability across cloud providers; rich ecosystem (Helm, operators, service mesh); native auto-scaling; rolling updates and canary deployments; large community support. Negative: Operational complexity (node management, security updates, networking); cost of control plane (\$72/month for EKS); team needs Kubernetes expertise.
Related Decisions	ADR-003 (API Gateway), ADR-006 (CI/CD Strategy)
Date	2025-03-19

Deliverable 6: Technology Stack Matrix

Component	Technology	Version	Justification	Alternatives Considered
Web Frontend	React.js	18.x	Strong ecosystem, component reusability, excellent developer experience, large talent pool	Angular (steeper learning curve), Vue (smaller ecosystem for enterprise)

Component	Technology	Version	Justification	Alternatives Considered
Mobile App	React Native	0.72	Code reuse with web (shared business logic), large community, good for scanning apps, cross-platform	Flutter (requires learning Dart), Native iOS/Android (no code reuse)
API Application	Python/Flask	2.3	Rapid development, excellent ML/AI libraries (scikit-learn, TensorFlow), team skills	Node.js (weaker ML ecosystem), Java/Spring (slower development, more boilerplate)
API Gateway	Kong	3.x	Open source, plugin ecosystem, JWT validation, rate limiting, runs on Kubernetes	AWS API Gateway (vendor lock-in), NGINX (less built-in features)
Core Database	PostgreSQL	14+	ACID compliance, JSON support, full-text search, mature, open source, Snipe-IT compatible	MySQL (weaker JSON support), SQL Server (licensing cost)
Document Store	MongoDB	6+	Flexible schema, good for	Couchbase (less adoption),

Component	Technology	Version	Justification	Alternatives Considered
			metadata, easy horizontal scaling, managed Atlas service	DynamoDB (vendor lock-in)
Time-Series DB	InfluxDB	2.x	Optimized for telemetry, high write throughput, native downsampling, Flux query language	TimescaleDB (PostgreSQL-based but higher overhead for time-series)
Graph Database	Neo4j	5.x	Best for relationship queries, Cypher query language, mature drivers, Aura managed service	Amazon Neptune (vendor lock-in), ArangoDB (less mature)
Cache	Redis	7.x	High performance, rich data structures (hashes, sets, sorted sets), widely adopted	Memcached (fewer data structures, no persistence)
Message Broker	Kafka	3.x	High throughput (millions msgs/sec),	RabbitMQ (lower throughput), AWS SQS (vendor lock-

Component	Technology	Version	Justification	Alternatives Considered
			durability (disk persistence), exactly-once semantics, ecosystem	in, limited ordering)
CDC	Debezium	2.x	Native PostgreSQL support (logical decoding), Kafka integration, schema evolution	Custom CDC (re-inventing wheel), AWS DMS (less flexible)
Workflow Engine	Temporal	1.20	Stateful workflows, durable execution, automatic retries, compensation support	Airflow (batch-oriented, not for long-running workflows), Camunda (Java-centric)
Batch Processing	Airflow	2.x	Mature DAG management, Python-native, large operator ecosystem, MWAA managed	Prefect (newer, less mature), Dagster (smaller community)
Search Engine	Elasticsearch	8.x	Full-text search, faceted search, analytics aggregations,	OpenSearch (forked from Elasticsearch, less ecosystem)

Component	Technology	Version	Justification	Alternatives Considered
			Elastic Cloud managed	
AI Model Serving	SageMaker	-	Integrated with AWS, managed endpoints, auto-scaling, A/B testing, monitoring	Custom container (more operational overhead), Lambda (cold start issues)
Container Orchestration	EKS (Kubernetes)	1.28	Portability, ecosystem, auto-scaling, service discovery, managed control plane	ECS (AWS-specific), Self-managed K8s (operational overhead)
Infrastructure as Code	Terraform	1.5	Cloud-agnostic, mature state management, large provider ecosystem, team familiarity	Pulumi (less mature, smaller community), CloudFormation (AWS-specific)
CI/CD	GitLab CI	-	Integrated with source control, flexible pipelines, built-in container registry	Jenkins (maintenance overhead), GitHub Actions (external repo)

Component	Technology	Version	Justification	Alternatives Considered
Monitoring	Prometheus + Grafana	-	Industry standard, flexible metrics, powerful queries, large ecosystem	Datadog (cost), CloudWatch (vendor lock-in, limited queries)
Logging	ELK Stack	8.x	Centralized logging, full-text search, visualization, Elastic Cloud managed	Splunk (cost), Loki (less mature, limited search)
Tracing	Jaeger	1.50	OpenTelemetry compatible, distributed tracing, root cause analysis	Zipkin (less features, less active development)

Deliverable 7: Traceability Matrix

Pain Point to Application Solution Traceability

Pain Point ID	Pain Point Description	Application Solution	C4 Level	Component
P1	Procurement-to-inventory delay (3-5 days)	Procurement API integration; automated asset creation from PO events	Container	API Application, Kafka, Procurement Client
P3	Poor mobile experience for scanning	React Native mobile app with offline scanning and barcode generation	Container	Mobile Application
P5	Software license over-purchasing	License Management Service with usage tracking and compliance dashboard	Component	License Service, License Repository
P7	Asset non-return after offboarding (30% loss rate)	HRIS integration; Temporal workflow for offboarding with escalation	Container	Workflow Orchestrator, HRIS Client
P9	Finance maintains	ERP API integration; automated depreciation export;	Component	Batch Processor (Airflow), Finance Client

Advanced System Design and Implementation

Pain Point ID	Pain Point Description	Application Solution	C4 Level	Component
	separate spreadsheets	Batch Processor for monthly reports		
P12	Audit findings on inventory accuracy	Graph database (Neo4j) for relationships; complete assignment history tracking	Container	Neo4j Client, Assignment Repository
P15	Predictive maintenance failures	Telemetry ingestion service; Predictive Analytics Service with SageMaker	Container	Telemetry Service, Predictive Service, InfluxDB

TO-BE Process to Application Support Traceability

TO-BE Process	Application Support	Containers Involved	Components
1.0 User Sync	HRIS webhook receiver; processes employee/student status changes	API Application, Kafka	User Controller, User Service, Kafka Producer
2.0 Procurement Intake	Procurement API integration; creates asset records from PO data	API Application, Kafka	Procurement Controller, Asset Service, Kafka Producer

TO-BE Process	Application Support	Containers Involved	Components
3.0 Asset Assignment	Assignment workflow with validation and history tracking	API Application, Temporal, PostgreSQL	Asset Service, Workflow Client, Assignment Repository
4.0 Telemetry Ingestion	MDM integration (JAMF/Intune); telemetry processing and storage	Telemetry Service, InfluxDB, Kafka	Telemetry Consumer, InfluxDB Repository, Kafka Consumer
5.0 AI Analytics	Predictive models for failure and demand forecasting	Predictive Service, Kafka, SageMaker	Forecasting Service, Anomaly Detection, Model Client
6.0 License Management	License allocation, compliance monitoring, renewal alerts	API Application, PostgreSQL	License Service, License Repository
7.0 Asset Retirement	Retirement workflow with data sanitization and disposal	Temporal, API Application, Batch Processor	Retirement Workflow, Asset Service, Export Service

Data Architecture to Application Mapping

Data Architecture Element	Application Support	Container	Component
Enterprise Asset Entity	Asset domain model with business behavior	API Application	Asset Entity, Asset Service

Data Architecture Element	Application Support	Container	Component
Snipe-IT Core Tables (PostgreSQL)	Asset repository implementation with CRUD operations	API Application	Postgres Asset Repository
Custom Assignment History Table	Assignment tracking and audit trail	API Application, PostgreSQL	Assignment Repository, Audit Service
Telemetry Time-Series (InfluxDB)	Telemetry ingestion and historical query	Telemetry Service, Predictive Service	InfluxDB Repository, Forecasting Service
Graph Network Relationships (Neo4j)	Network mapping queries and relationship traversal	API Application, Neo4j	Neo4j Client, Network Service
CDC Event Stream (Debezium + Kafka)	Change propagation from PostgreSQL to other stores	Debezium, Kafka	Kafka Producer, Sync Consumers
Metadata Document Store (MongoDB)	Warranty documents, configuration files, images	API Application	MongoDB Repository